

Lecture 3: STM32 Interrupt and ISR

18 조 – 이해리, 장동화

I. INTRODUCTION

A. 실험 목적

이번 실험을 통해 STM32 마이크로프로세서에서의 인터럽트와 ISR에 대해 익혀보았다. 컴퓨터는 C 언어와 일대일 대응인 컴파일된 이진 기계를 프로그램 카운터(PC)에 의해 순서대로 실행한다. 따라서 모든 컴퓨터가 하는 본질은 조건문을 반복하며 어딘가에 읽고 쓰는 것이다. 또한 중앙연산유닛(CPU)은 그 연산능력을 극대화하기 위해 시스템에서 가장 빠른 속도(수십 MHz~ 수 GHz)로 작동한다.

앞서 설명한대로 어떤 순간에서의 CPU는 이미 결정된 기계어와 PC에 의해 다음에 실행할 명령어가 완전히 정해져 있다. 그러나 외부세계의 이벤트는 매우 다양하고 비결정적이거나 CPU에 비해 느리기 때문에 단순히 반복문(Polling)만으로는 CPU의 빠른 연산능력을 효율적으로 활용하면서 외부와 정보를 주고받을 수 없다.

프로세서가 인터럽트 신호를 감지하면, 처리중인 명령어를 완료하고 다음으로 실행할 명령의 주소와 복귀에 필요한 정보들을 잠시 저장해두고 PC에 ISR의 주소를 강제로 기입한다.[1] 루틴이 끝나며 원래 상태로 완전히 돌아온다.

B. 실험 과정

지난 실습에서 Polling 방식을 통한 버튼 상태 읽기를 통해 LED를 제어하였다. 이번 실습에서는 인터럽트를 통해 버튼과 외부 GPIO 핀 상태 변화를 감지하여 LED를 제어해보고자 한다. Polling 방식을 통한 변화 감지는 굉장히 비효율적이다. HAL_Delay 함수는 ms 단위의 정수를 받아서 내부적으로 while 문을 돌리는 구조이다.

```
371 _weak void HAL_Delay(uint32_t Delay)
372 {
373     uint32_t tickstart = HAL_GetTick();
374     uint32_t wait = Delay;
375
376     /* Add a freq to guarantee minimum wait */
377     if (wait < HAL_MAX_DELAY)
378     {
379         wait += (uint32_t)(uwTickFreq);
380     }
381
382     while ((HAL_GetTick() - tickstart) < wait)
383     {
384     }
385 }
386
```

Src/stm32f1xx_hal.c:371

```
284 /**
285  * @brief This function is called to increment a global variable "uwTick"
286  *        used as application time base.
287  * @note In the default implementation, this variable is incremented each 1ms
288  *        in SysTick ISR.
289  * @note This function is declared as __weak to be overwritten in case of other
290  *        implementations in user file.
291  * @retval None
292  */
293 _weak void HAL_IncTick(void)
294 {
295     uwTick += uwTickFreq;
296 }
```

Src/stm32f1xx_hal.c:293

이러한 구조는 초등 MCU인 아두이노에서 많이 쓰이는 방식인데, HAL_GetTick 함수는 또한 내부적으로는 ISR를 사용하는데, uwTick 변수는 SysTick ISR에 의해 1ms 마다 증가하며 이를 단순 비교/반복하여 딜레이를 구현한다.

II. EXPERIMENTAL RESULTS

A. Experiment #1- 버튼/함수발생기 인터럽트 구현

STM32 MCU의 EXTI(EXTernal Interrupt) ISR (Interrupt Service Routine)는 다음과 같이 구성된다. ST에서 이미 작성해둔 어셈블리 코드(startup_stm32f103xb.s)를 살펴보면 EXTIX_IRQHandler라는 변수명(함수명)을 지정해둔 것을 알 수 있다. 이를 통해 링커가 사전 컴파일된 ABI 라이브러리에 함수 주소를 링킹해두면 자동으로 인터럽트 신호가 들어왔을 때 해당하는 함수의 주소가 실행되게 된다. ISR은 EXTIX_IRQHandler를 거쳐 HAL_GPIO_EXTI_

IRQHandler, HAL_GPIO_EXTI_Callback 함수를 차례로 실행하고 끝나게 된다. 사용자 코드는 서로 다른 종류의 MCU 간의 코드 재사용성을 위해 HAL 함수에 작성하는 것이 권장된다. STM32CubeMX 에서 GPIO 핀에 EXTI 를 상승에지/하강에지 중 하나로 선택, NVIC 에서 해당 EXTIX 를 사용할지 여부, 우선순위 등을 선택하고 Generate Code 를 하면 HAL_GPIO_EXTI_Callback 함수보다 덜 추상화된 내부 로직 등은 자동으로 생성된다. 코드가 생성된 후 기존의 비효율적인 코드인 Polling 방식 while 반복문은 제거하고, 다음과 같이 미리 선언된 HAL 함수만 정의해주면 된다.

```
221 /* USER CODE BEGIN 4 */
222+ void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
223+   if(GPIO_Pin == Switch_interrupt_harry_Pin){
224+       PC13_Counter++;
225+       HAL_GPIO_TogglePin(LED2_harry_GPIO_Port, LED2_harry_Pin);
226+   }
227+ }
228 /* USER CODE END 4 */
```

함수발생기 인터럽트에 의한 LED 제어 방식도 코드는 동일하다. 다만 Switch 변수에 해당하는 핀만 STM32CubeMX GUI 상에서 내가 함수발생기 입력을 받아올 핀으로 변경해주면 된다. 여기서 주의해야 할 것은, 스위치는 누르면 리셋이기 때문에 핀 pull-up 에 인터럽트 감지에 falling-edge 를 사용, 함수발생기의 경우 동일하게 하여도 작동은 하겠으나 의도상 pull-down 에 rising-edge 를 사용해주는 것이 좋다.

B. Experiment #2- 인터럽트간 우선순위와 Nested Interrupt

이번 실습을 통해 서로 다른 우선순위의 인터럽트 3 개가 중첩되어 실행되는 과정을 살펴볼 수 있었다. ISR 은 현재 실행중인 코드를 중단하고 잠깐 실행하는 행동이다. 또한 결정적인 코드를 실행하다가 임의의 비결정적인 순간에 다른 위치로 이동하여 실행하는 것이기 때문에 ISR 의 실행시간이 길어진다면 시스템 전체의 결정성을 보장하기 힘들어진다. 여러가지 외부 기기가 있고 이들이 각각 인터럽트 신호를 CPU 에 전달하기에 인터럽트 간의 우선

순위도 또한 필요하다. ISR 실행 중에 더 우선순위가 높은 다른 인터럽트가 발생하여 해당하는 ISR 로 넘어가는 것을 Nested Interrupt 라고 하는데, ARM Cortex-M3 에는 별도의 인터럽트 컨트롤러(NVIC)를 하드웨어로 내장하고 있기 때문에[2] 기본적으로 interrupt nesting 을 지원한다. 이번 실습에서는 다음과 같이 중첩된 ISR 를 구현해보았다.

[스위치 인터럽트 -> GPIO_C1 출력 [GPIO_C8 입력 감지 인터럽트 -> LED 250ms 간격 깜빡이다 중간에 GPIO_C0 출력[GPIO_B1 입력 감지 인터럽트-> LED 500ms 간격 2 번 깜빡임]->250ms 간격으로 마저 깜빡임]]

여기서 주의할 점은, HAL_Delay 함수의 사용이 ISR 도중에는 제한된다는 점이다. HAL_Delay 함수는 내부적으로 SysTick 인터럽트를 사용하는데, 이 인터럽트는 우선순위가 기본값 15 로 돼있기 때문이다. 또한 우리가 사용할 3 개의 인터럽트의 경우에도 원래는 기존 인터럽트 우선순위가 있지만, STM32CubeMX GUI 상 입력돼있는 우선순위 0 을 별도로 수정하지 않는다면 ISR 도중 다른 인터럽트가 발동되지 않는다. 같은 우선순위의 인터럽트는 현재 인터럽트가 종료된 후에 실행되기 때문이다. 이번 실습의 구현 의의는 ISR 실행 도중에 더 우선순위인 인터럽트가 발생하는 것을 관찰하는 것이기에, ISR 내부에서 사용할 Delay 함수를 다음과 같이 별도로 작성하였다.

```
244 /* USER CODE BEGIN 4 */
245 void Delay(uint32_t Delay)
246 {
247     while(Delay) {
248         Delay--;
249     }
250 }
251
252 void Delay_50ms(float N){
253     Delay(Delay: (uint32_t)(265967*N));
254 }
```

그리고 ISR Callback 부분은 다음과 같이 작성해주었다.

```

256 void LED_250ms(){
257     int i=0;
258     while (i<40) {
259         Delay_50ms(N: 5);
260         i++;
261         HAL_GPIO_TogglePin(GPIOx: LED2_harry_GPIO_Port, GPIO_Pin: LED2_harry_Pin);
262         HAL_GPIO_TogglePin(GPIOx: PC3_harry_GPIO_Port, GPIO_Pin: PC3_harry_Pin);
263         if (i==20){
264             HAL_GPIO_TogglePin(GPIOx: PC0_harry_GPIO_Port, GPIO_Pin: PC0_harry_Pin);
265             HAL_GPIO_TogglePin(GPIOx: PC0_harry_GPIO_Port, GPIO_Pin: PC0_harry_Pin);
266         }
267     }
268 }
269
270 void LED_500ms(){
271     int i=0;
272     while (i<4) {
273         Delay_50ms(N: 10);
274         i++;
275         HAL_GPIO_TogglePin(GPIOx: LED2_harry_GPIO_Port, GPIO_Pin: LED2_harry_Pin);
276         HAL_GPIO_TogglePin(GPIOx: PC3_harry_GPIO_Port, GPIO_Pin: PC3_harry_Pin);
277     }
278 }
279
280 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
281     if(GPIO_Pin == PC13_harry_Pin){
282         PC13_Counter++;
283         HAL_GPIO_TogglePin(GPIOx: PC1_harry_GPIO_Port, GPIO_Pin: PC1_harry_Pin);
284     }
285 }
286
287 if(GPIO_Pin == PC8_harry_Pin){
288     LED_250ms();
289 }
290
291 if(GPIO_Pin == PB1_harry_Pin){
292     LED_500ms();
293 }
294 }

```

코드를 실행한 결과 의도한대로 스위치를 누르면 10 번 빠르게 깜빡인 후 2 번 천천히 깜빡이고 다시 10 번 빠르게 깜빡이는 것을 확인할 수 있다. 스위치를 여러번 눌러도 마찬가지로 스위치 누르는 인터럽트끼리의 우선순위가 같기 때문에 현재 ISR 다발이 모두 실행된 후에 다음 ISR 다발이 실행되는 것도 확인할 수 있었다.

III. CONCLUSION

인터럽트와 인터럽트 서비스 루틴에 대해 익히고 이해하기 위해 두 실습을 해보았다. 첫번째 실습을 통해 인터럽트 기반 LED 제어, 두번째 실습을 통해 인터럽트 중첩과 인터럽트 우선순위를 각각 익힐 수 있었다. 두번째 실습에서 여러가지 착오사항이 있었는데 다음과 같은 착오였다. CubeMX Gui 상 우선순위를 지정해주지 않으면 EXTI4 의 기본 우선순위인 4 가 아닌 최우선순위 0 이 들어가기 때문에 서로 Nesting 할 수 없었고, ISR 도중 더 우선순위의 ISR 이 시작되는 것을 관찰하지 못하였다. HAL_Delay 함수는 가장 낮은 우선순위(15) 인터럽트를 통해 SysTick 을 받아오기 때문에, ISR 도중에는 사용이 불가능하였다. 또한 도달할 수 없는 부분의 코드가 실행돼야 한다고 착오를 한 부분이 있어서 시행착오를 거쳐 해결하였다.

REFERENCES

- [1] James E. Smith and A. R. Pleszkun, “Implementing Presice Interrupts in Pipelined Processors”, IEEE Transactions on Computers, Vol. 37, No. 5, May 1988, pp. 202-213
- [2] <https://developer.arm.com/documentation/100165/0201/Nested-Vectored-Interrupt-Controller/NVIC-functional-description/NVIC-interrupts>